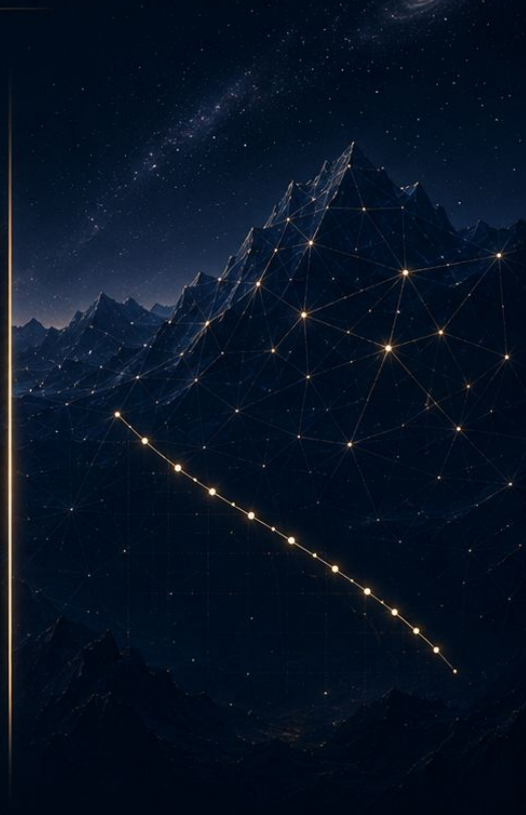
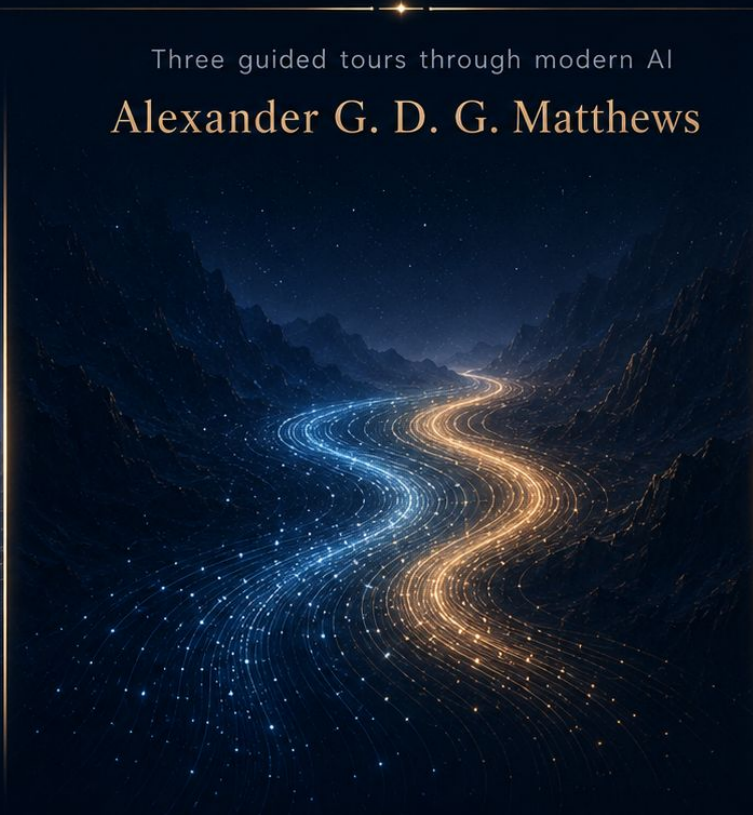
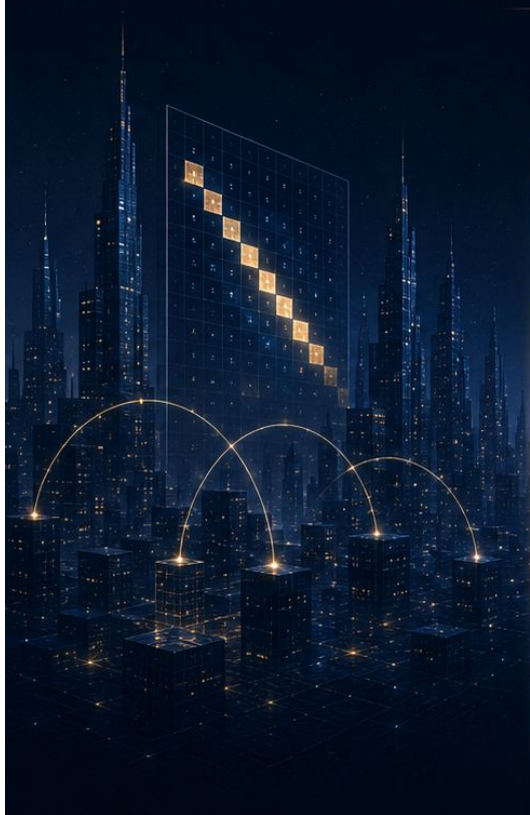


Concepts at the AI–Physics Interface

Three guided tours through modern AI
Alexander G. D. G. Matthews



Plan for lecture series

1) **A Hitchhiker's Guide to Transformers and Attention**

Introduction to some basic concepts useful throughout the lecture series.

Introduce a crucial architecture that everyone should know.

2) **Probability transport and flow matching, with connections to diffusion models**

Crucial idea in generative modelling of natural images and science foundation models.

Beautiful mathematics with a lovely range of connections across many fields.

3) **Random neural networks, training dynamics, and scaling behaviour**

Crucial theoretical ideas in the training and scaling of neural networks.

Strong connections to ideas from statistical physics.

A Hitchhiker's Guide to Transformers and Attention

Alexander G. D. G. Matthews

✦ Concepts at the AI-physics interface ✦



Plan for this lecture

1) Foundational concepts

- Regression and classification

- Maximum likelihood and compression

- Multi-layer perceptrons and building representations

- Autoregression

2) Fundamentals of transformer and self-attention

- Concept of attention

- Transformer translation architecture

- Vision transformer

3) After the transformer

- Chat-GPT and dialogue agents

- Scaling

- AlphaFold II

Plan for this lecture

1) Foundational concepts

- Regression and classification

- Maximum likelihood and compression

- Multi-layer perceptrons and building representations

- Autoregression



2) Fundamentals of transformer and self-attention

- Concept of attention

- Transformer translation architecture

- Vision transformer

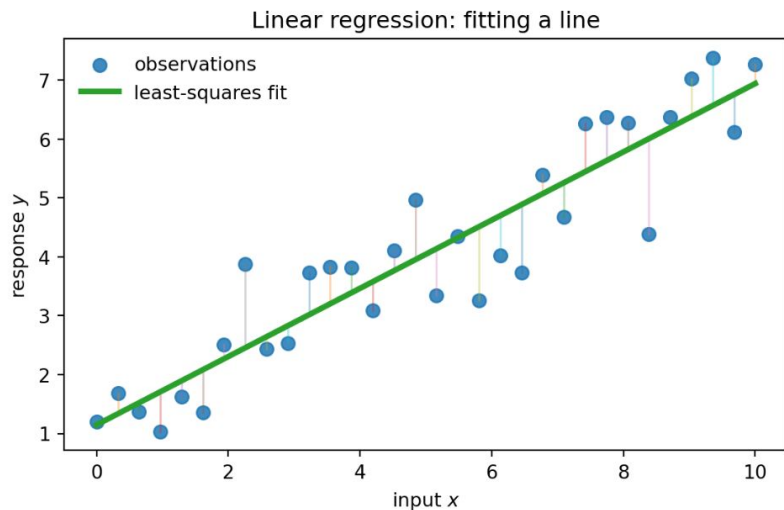
3) After the transformer

- Chat-GPT and dialogue agents

- Scaling

- AlphaFold II

Linear regression: least squares fit



Model:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$

Least-squares objective:

$$\hat{\beta} = \arg \min_{\beta} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

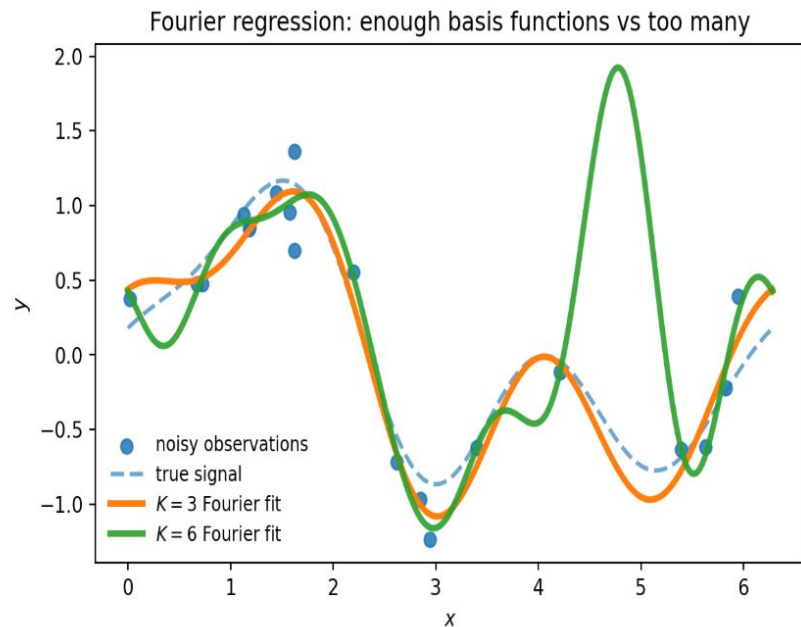
Matrix solution:

$$\hat{\beta} = (X^{\top} X)^{-1} X^{\top} y$$

Residuals:

$$r_i = y_i - \hat{y}_i$$

Linear basis regression



Linear regression generalises by replacing (x) with basis functions:

$$f_{\theta}(x) = \sum_{j=1}^p \theta_j \phi_j(x).$$

$$f_{\theta}(x) = a_0 + \sum_{k=1}^K [a_k \sin(kx) + b_k \cos(kx)].$$

Softmax: from logits to probabilities

Given logits $(z_1, \dots, z_N) \in \mathbb{R}^N$, softmax maps them to probabilities:

$$p_i = \text{softmax}_\beta(z)_i = \frac{\exp(\beta z_i)}{\sum_{j=1}^N \exp(\beta z_j)}.$$

$$p_i \geq 0, \quad \sum_{i=1}^N p_i = 1.$$

The inverse temperature β controls sharpness:

$$\beta = 0 \quad \Rightarrow \quad p_i = \frac{1}{N},$$

$$\beta \rightarrow \infty \quad \Rightarrow \quad p_i \rightarrow \begin{cases} 1, & i = \arg \max_j z_j, \\ 0, & \text{otherwise.} \end{cases}$$

Principle of maximum likelihood

Maximize the log-probability (density) of your predictions on the data.

$$\theta^* = \max_{\theta} \log p(\mathbf{y}|\theta)$$

Has a compression interpretation under a true source distribution $s(\mathbf{y})$.

$$-\mathbb{E}_{s(\mathbf{y})} [\log p(\mathbf{y}|\theta)] = H[s(\mathbf{y})] + \mathcal{KL}[s(\mathbf{y})||p(\mathbf{y}|\theta)]$$

For independent models $\log p(\mathbf{y}|\theta) = \sum_{i=1}^N \log p(y_i|\theta)$.

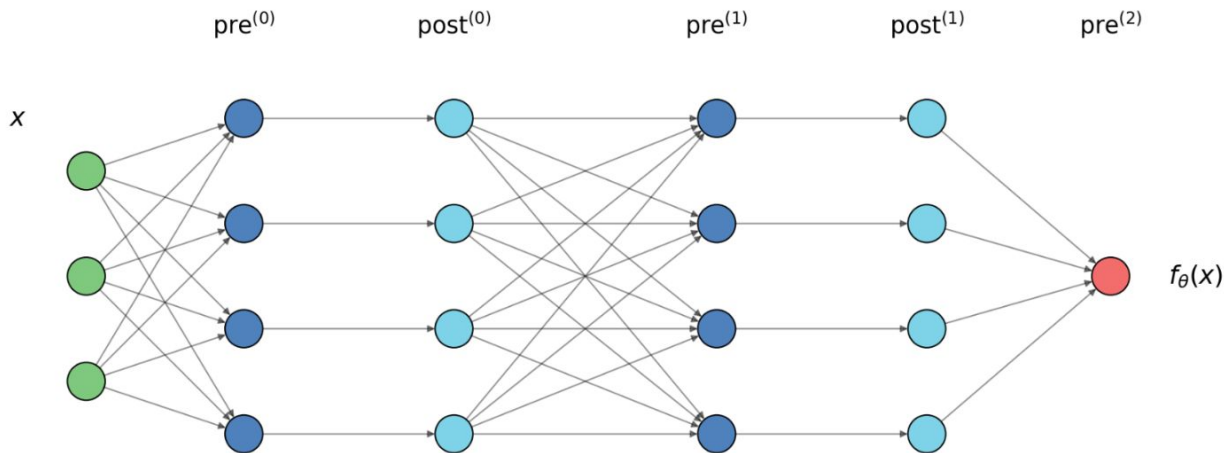
For regression we condition on inputs $p(\mathbf{y}|\mathbf{x})$.

ML can overfit when the model has the ability to do so.

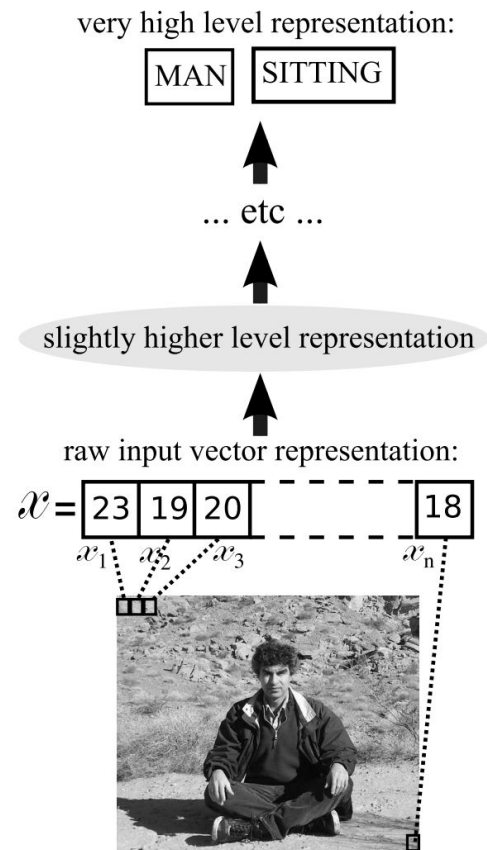
Multi-layer perceptron (MLP)

The network has width H and depth D , with nonlinearity ϕ .

$$f_i^{(\mu)}(x) = \sum_j w_{i,j}^{(\mu)} g_j^{(\mu)}(x) + b_i^{(\mu)}, \quad g_i^{(\mu+1)}(x) = \phi(f_i^{(\mu)}(x))$$



Representation learning



Autoregressive modelling

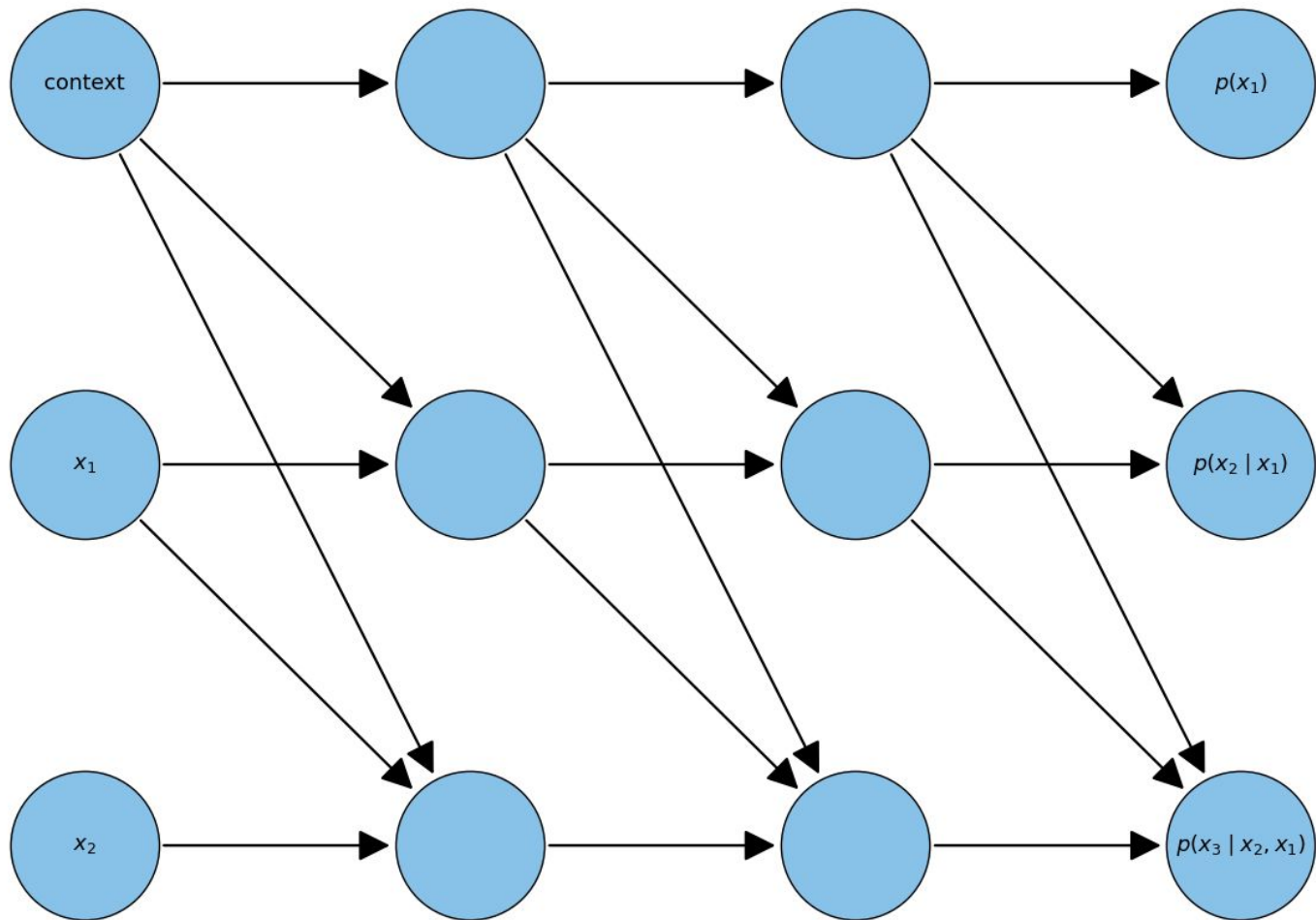
Chain rule of probability

$$p(x) = \prod_{i=1}^D p(x_i | x_{<i})$$

$$\log p(x) = \sum_{i=1}^D \log p(x_i | x_{<i})$$

Example:

$$p(x_3, x_2, x_1) = p(x_3 | x_2, x_1) p(x_2 | x_1) p(x_1)$$



Autoregressive/causal masking

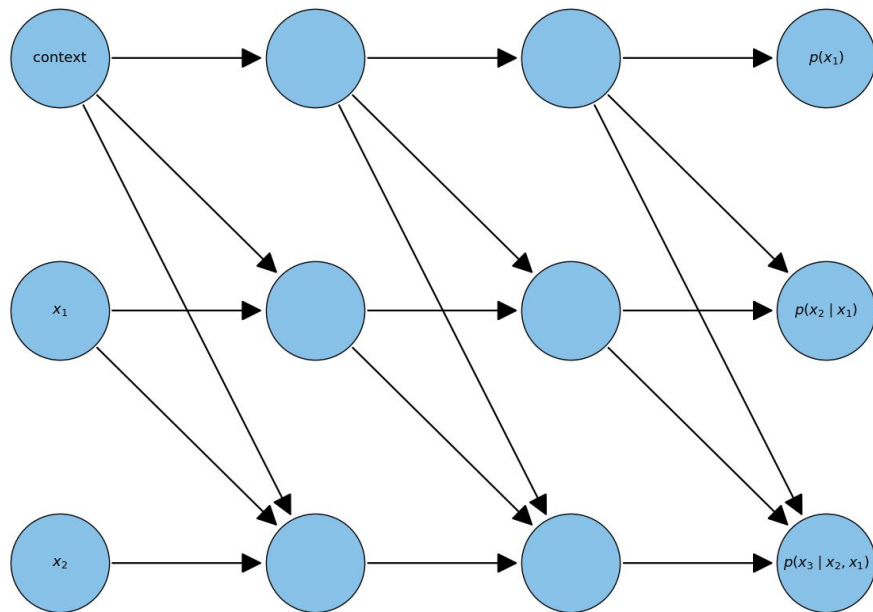
Chain rule of probability

$$p(x) = \prod_{i=1}^D p(x_i | x_{<i})$$

$$\log p(x) = \sum_{i=1}^D \log p(x_i | x_{<i})$$

This is like classifying the next token based on the previous ones.

Masked networks



Transformers

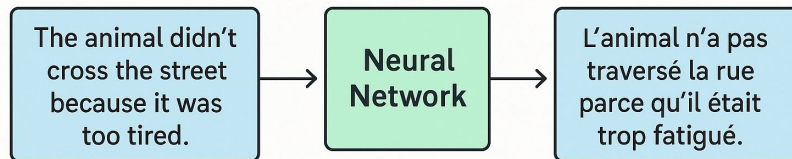
A **transformer** is a neural network **architecture** that was originally used for machine translation.

One of the innovative components was the use of particular types of **attention layers**.

The architecture is versatile. It has since been used for a huge variety of applications.

Notably a "generative" transformer is the crucial component of large language models (LLMs).

We will start by explaining the original translation case and then show how it can be used in vision.



Plan for this lecture

1) Foundational concepts

- Regression and classification
- Maximum likelihood and compression
- Multi-layer perceptrons and building representations
- Autoregression

2) Fundamentals of transformer and self-attention

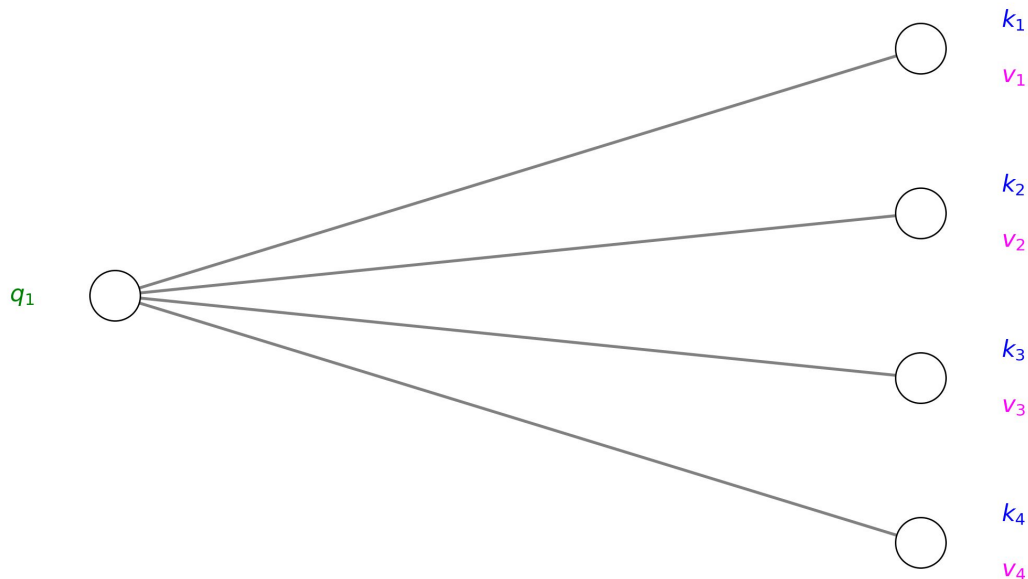
- Concept of attention
- Transformer translation architecture
- Vision transformer



3) After the transformer

- Chat-GPT and dialogue agents
- Scaling
- AlphaFold II

Attention



Query q_i

Key k_j

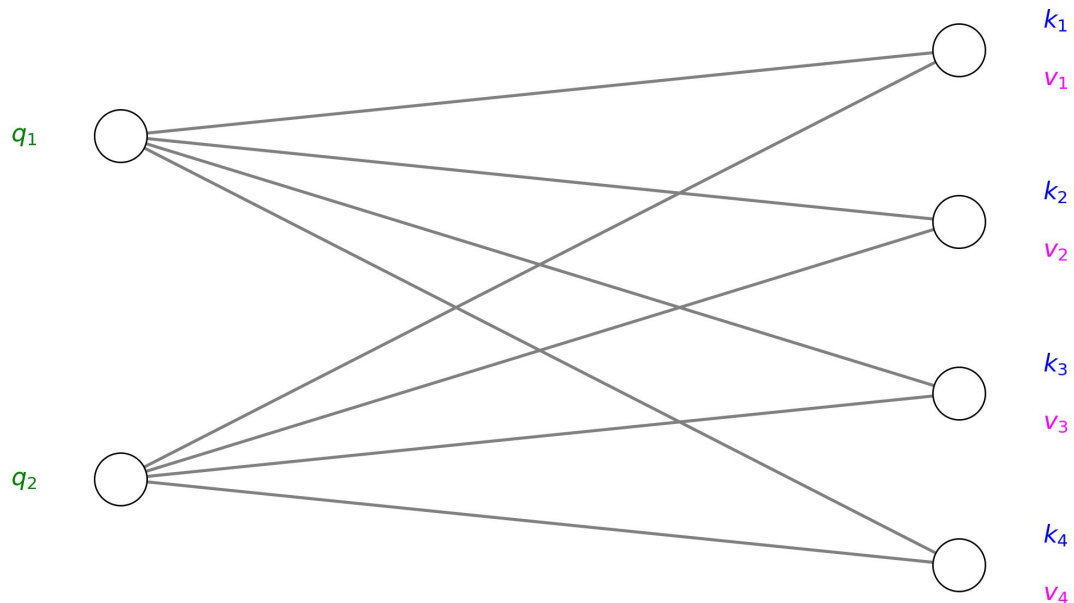
Value v_j

Compare the query to the key using $b_{i,j} = q_i \cdot k_j$.

Normalize using the softmax to give attention weight $a_{i,j} = \frac{\exp(b_{i,j})}{\sum_k \exp(b_{i,k})}$.

Take the weighted sum of the values $o_i = \sum_j a_{i,j} v_j$.

Attention



Query q_i

Key k_j

Value v_j

Compare the query to the key using $b_{i,j} = q_i \cdot k_j$.

Normalize using the softmax to give attention weight $a_{i,j} = \frac{\exp(b_{i,j})}{\sum_k \exp(b_{i,k})}$.

Take the weighted sum of the values $o_i = \sum_j a_{i,j} v_j$.

Attention: three types

1. Self-attention.

2. Masked self-attention.

3. Cross-attention.

Depends on how we choose q , k and v .

Query q_i

Key k_j

Value v_j

Compare the query to the key using $b_{i,j} = q_i \cdot k_j$.

Normalize using the softmax to give attention weight $a_{i,j} = \frac{\exp(b_{i,j})}{\sum_k \exp(b_{i,k})}$.

Take the weighted sum of the values $o_i = \sum_j a_{i,j} v_j$.

Tokens

The smallest unit of information the model will "see" in the input and output sequences.

The tokens could be words, characters or "sub-words". We assume words for text in this talk.

There is a special "terminal" token that tells generation when to stop.

There are "tokenizers" that convert text to tokens using compression ideas.

More generally tokens could be compressed chunks of audio or images. We will get to this later.

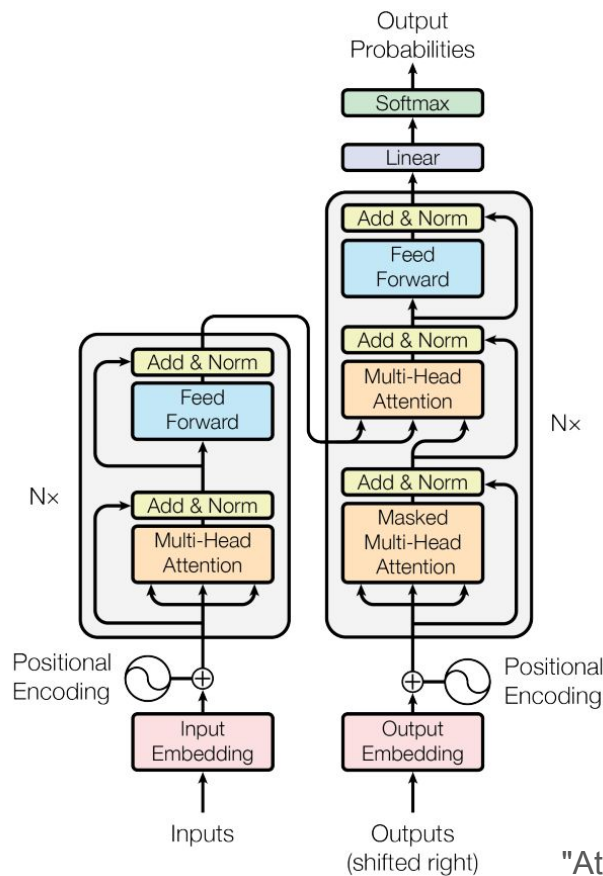
Translation architecture

We will break it down into parts.

The left hand side encodes the input.

The right hand side sequentially decodes (generates) the output.

The arrows between the sides are how the "meaning" is passed.

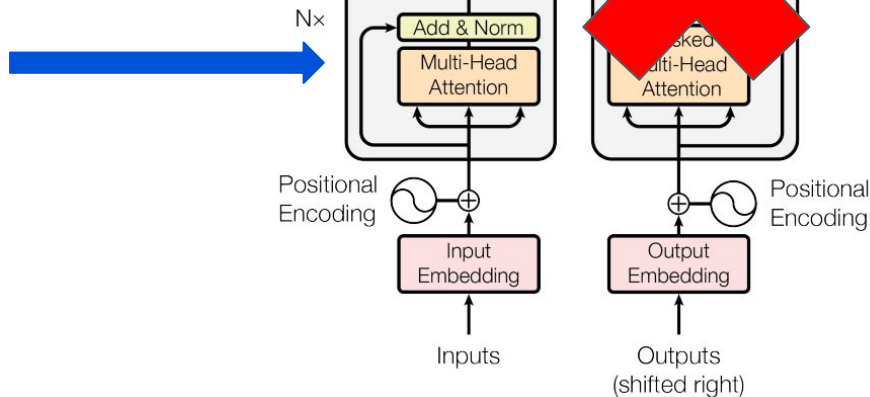


"Attention is all you need"
Vaswani et al. 2017

Pure encoder transformers

No decoder just an encoder.

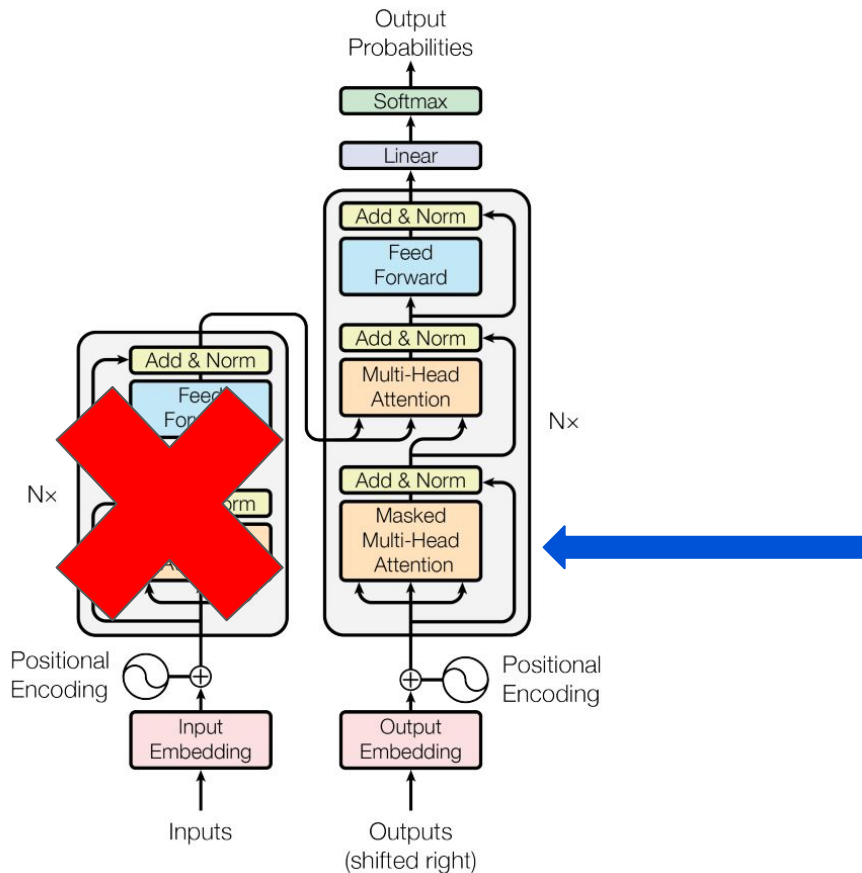
Extract some "meaning" from a sequence.



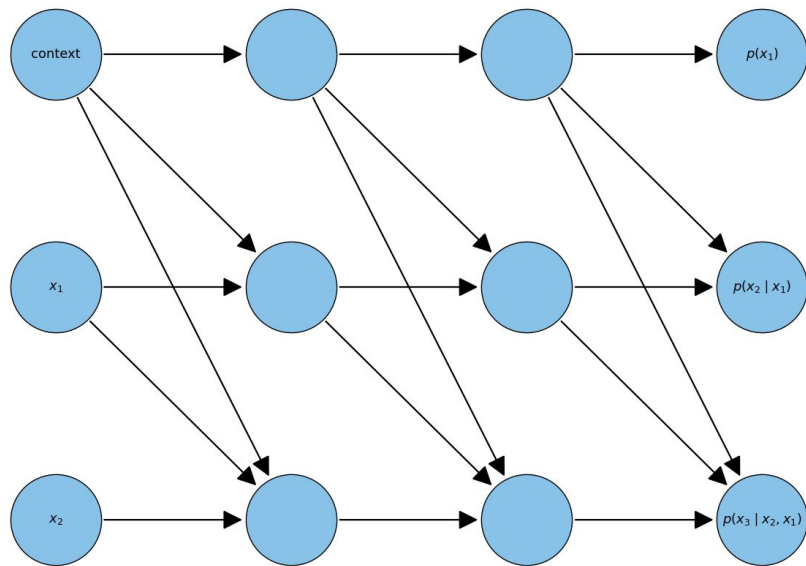
Pure generative transformers

No encoder just a decoder.

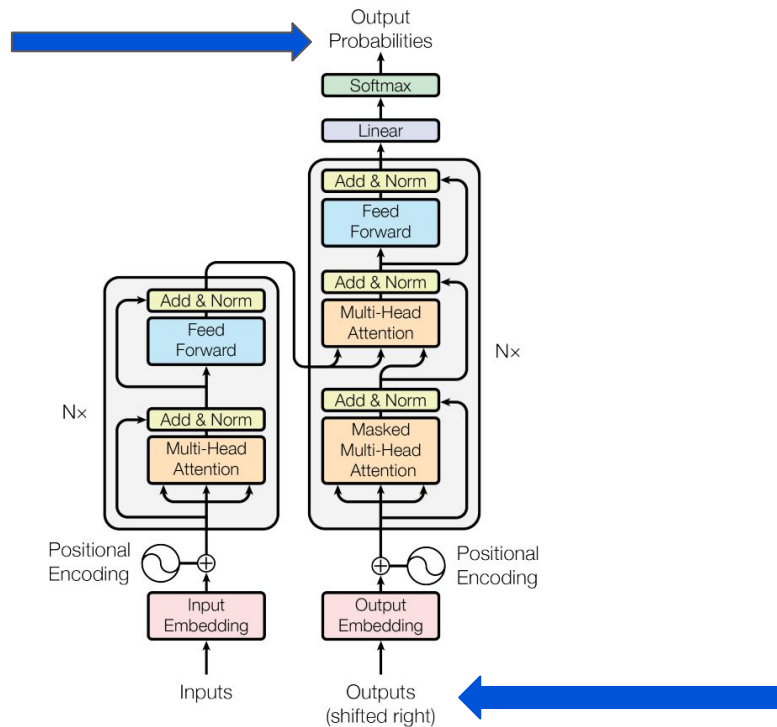
Effectively this is then
sequence completion.



Autoregressive/causal masking



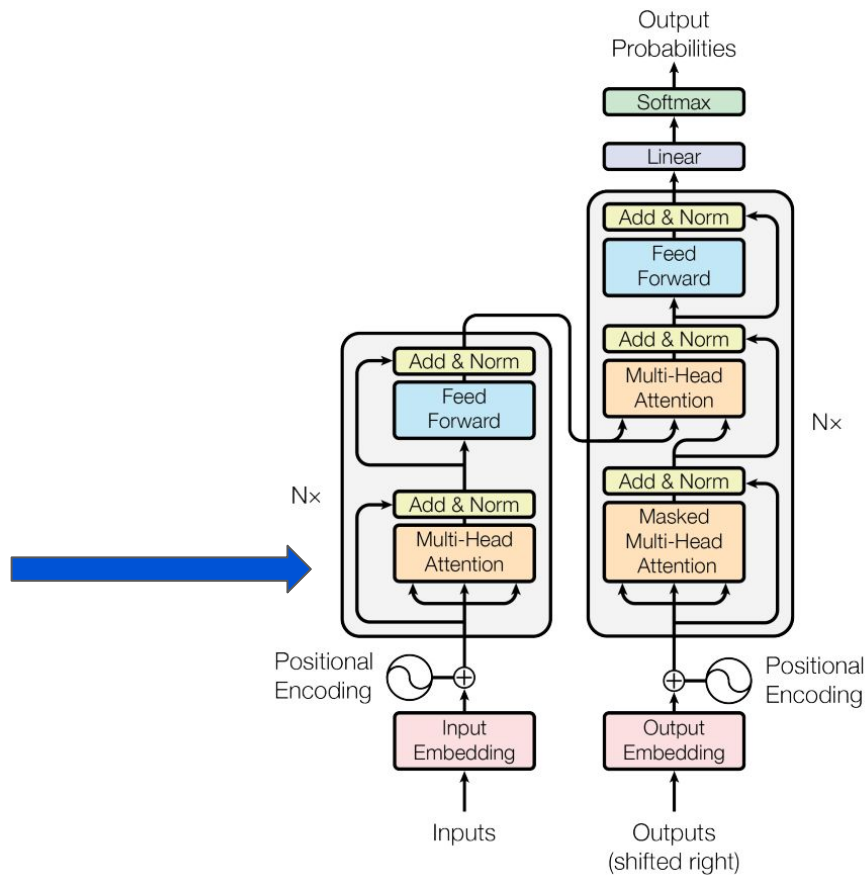
Training uses "offset by one" of output.



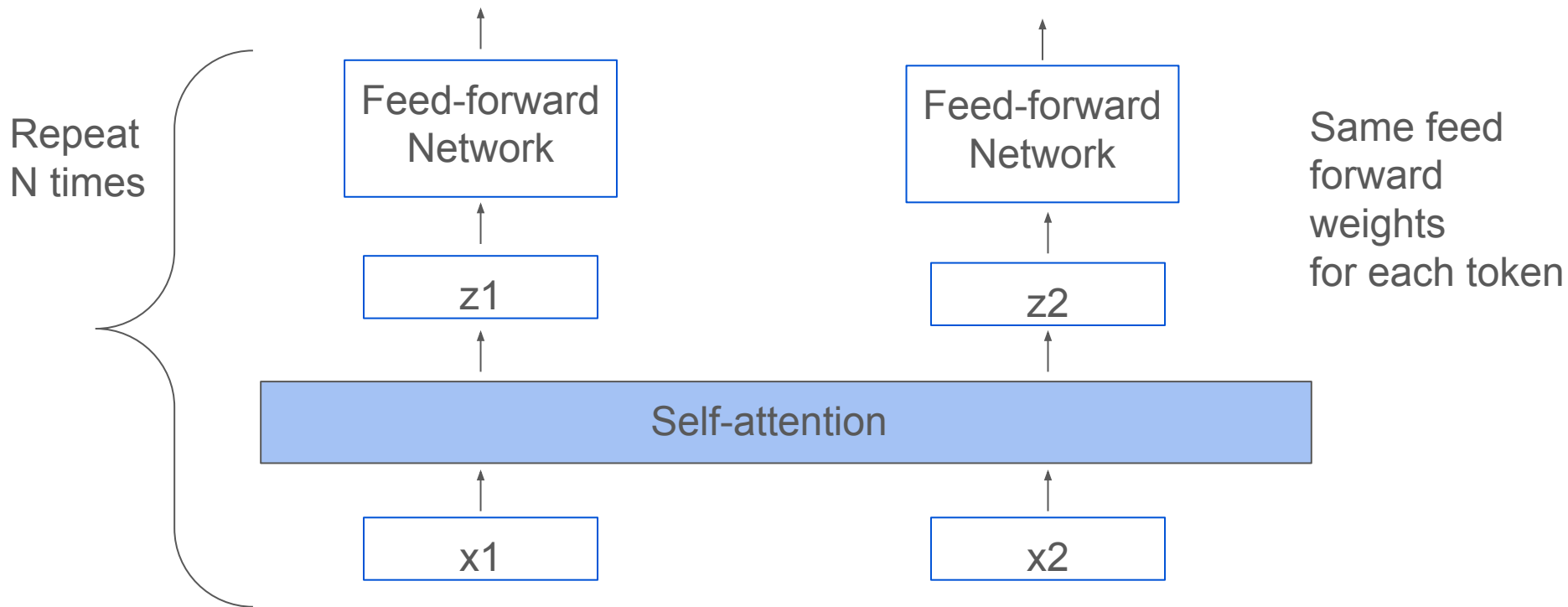
Generation uses samples from output and feeds it back in.

Translation architecture

Now will give details of the internals of the encoder layers.



Build a representation of tokens in a sequence



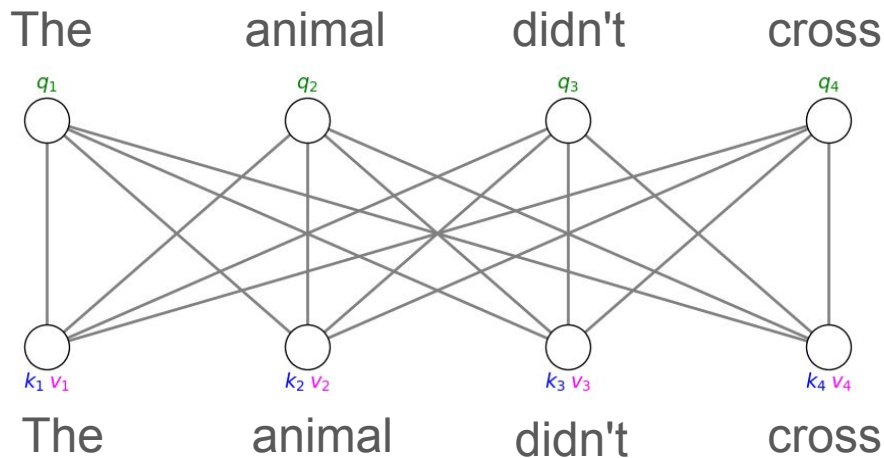
Self-Attention

Query, key and value are each a learnt transformation of the same input.

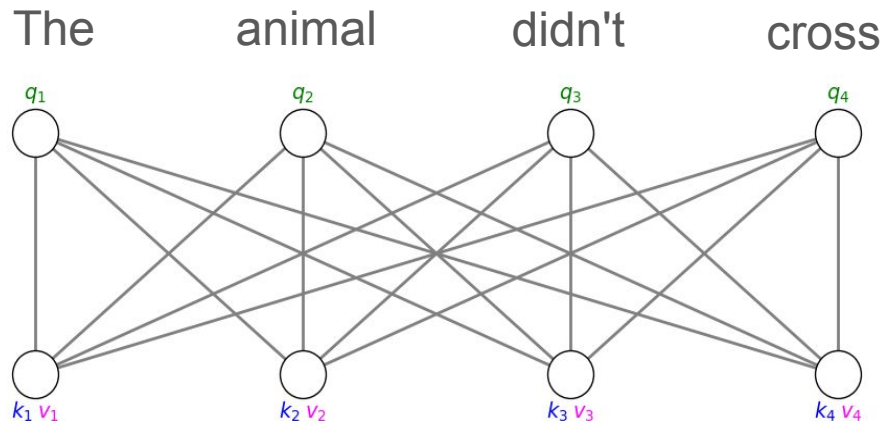
Query $q_i = W_q x_i + b_q$

Key $k_i = W_k x_i + b_k$

Value $v_i = W_v x_i + b_v$



Self-Attention



Query q_i

Key k_j

Value v_j

Compare the query to the key using $b_{i,j} = q_i \cdot k_j$.

Normalize using the softmax to give attention weight $a_{i,j} = \frac{\exp(b_{i,j})}{\sum_k \exp(b_{i,k})}$.

Take the weighted sum of the values $o_i = \sum_j a_{i,j} v_j$.

Multihead self-attention

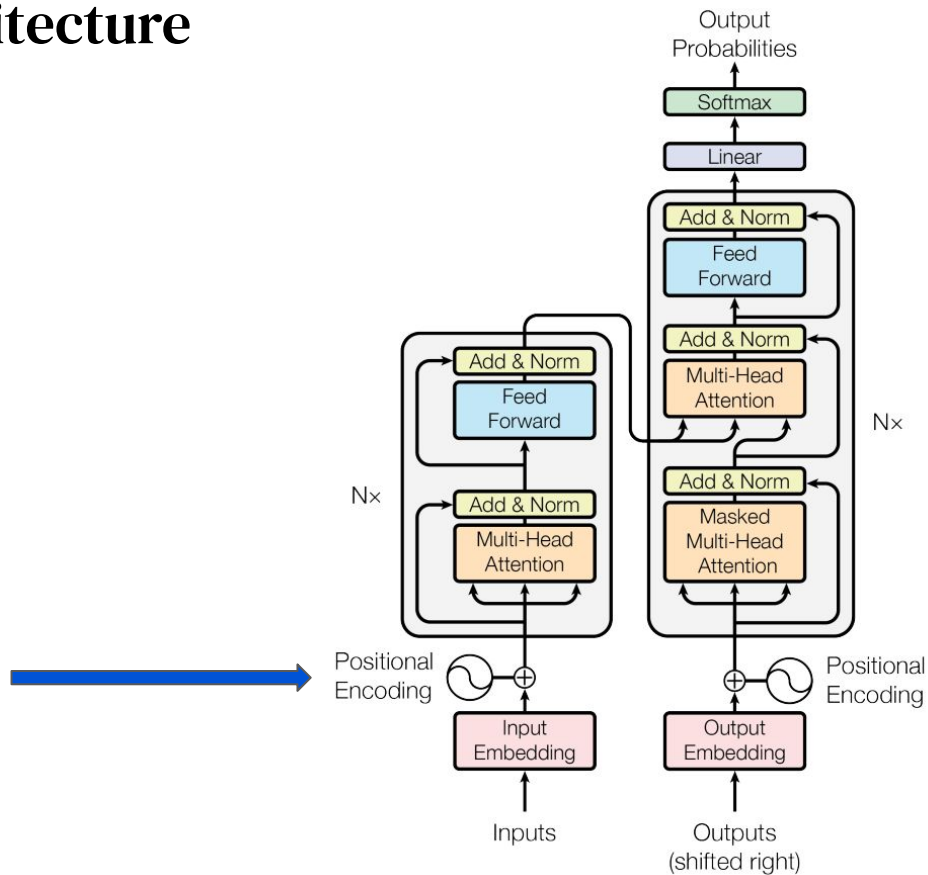
Words may relate to each other in multiple different ways.

E.g. a tree is green and it is a plant.

Do this by running the self-attention many times in parallel.

This is better than a single large attention layer precisely because it allows multiple types of relation.

Translation architecture



Position embeddings

The transformer encoder does not have a notion of sequence in the network structure.

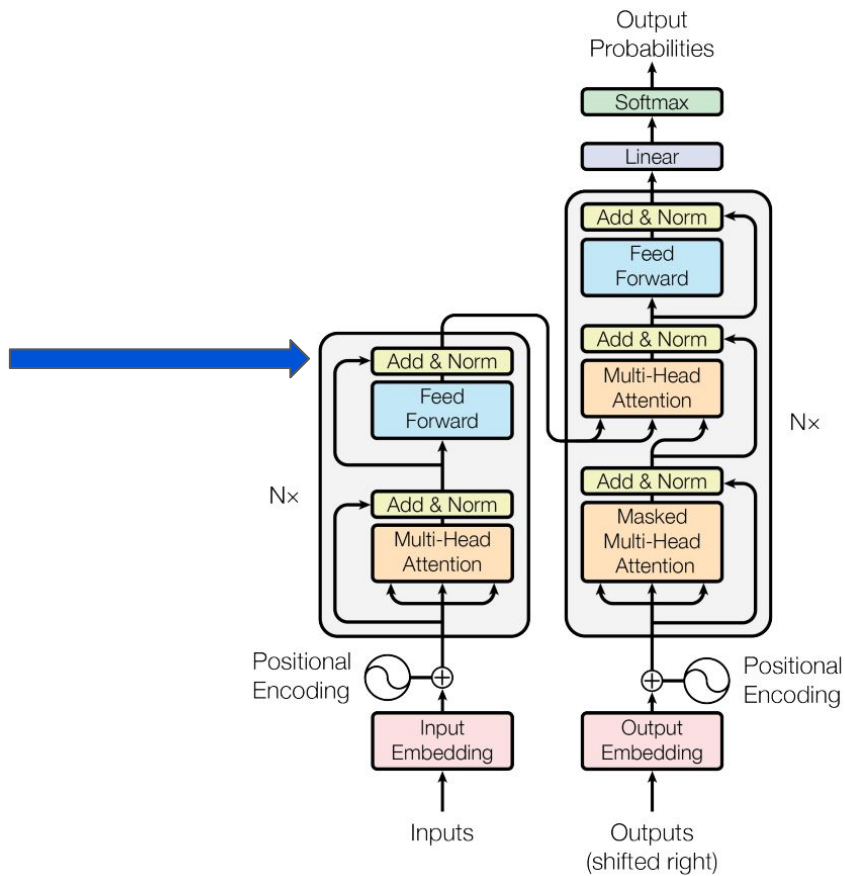
Instead the token position is appended as some additional features using sinusoids.

$$[\sin(\omega t), \cos(\omega t)]$$

Frequencies

Position in sequence

Translation architecture



Residual connections

Instead of learning the representation learn a change/update to the representation.

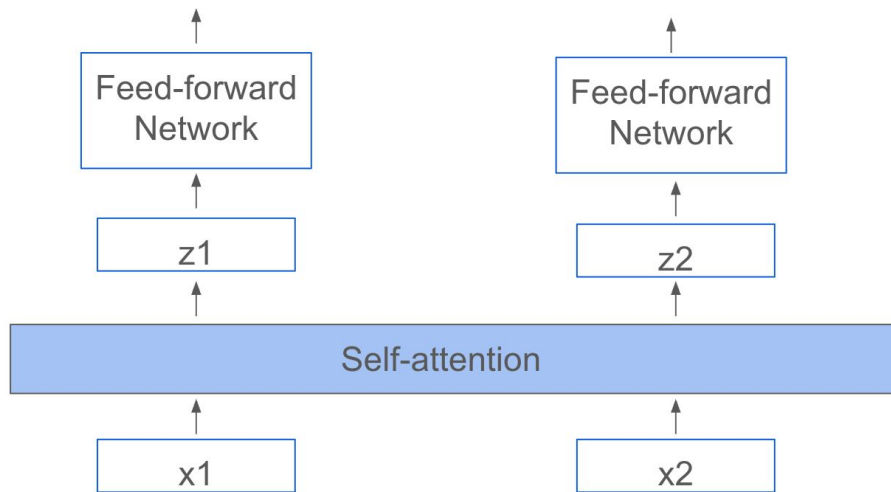
$$x_n = F_\theta(x_{n-1})$$

$$x_n = G_\theta(x_{n-1}) + x_{n-1}$$

It is important in practice for training deeper networks.

[Norm helps stabilize the magnitude of the representations so they don't explode or collapse]

Scaling of parameters and compute

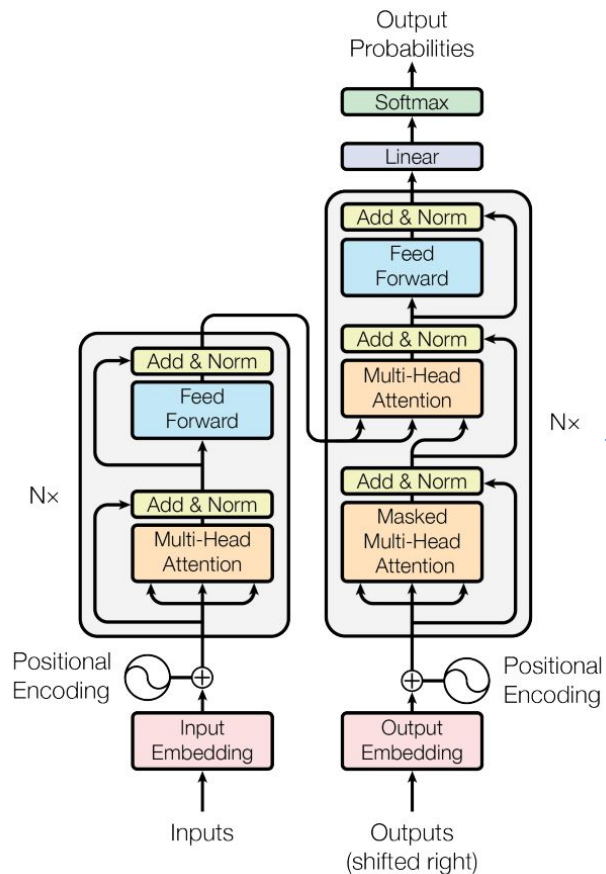


For fixed representation size and as a function of sequence length T :

- 1) $O(T^2)$ in flops coming from self-attention.
- 2) $O(1)$ parameters in T since the weights are shared.

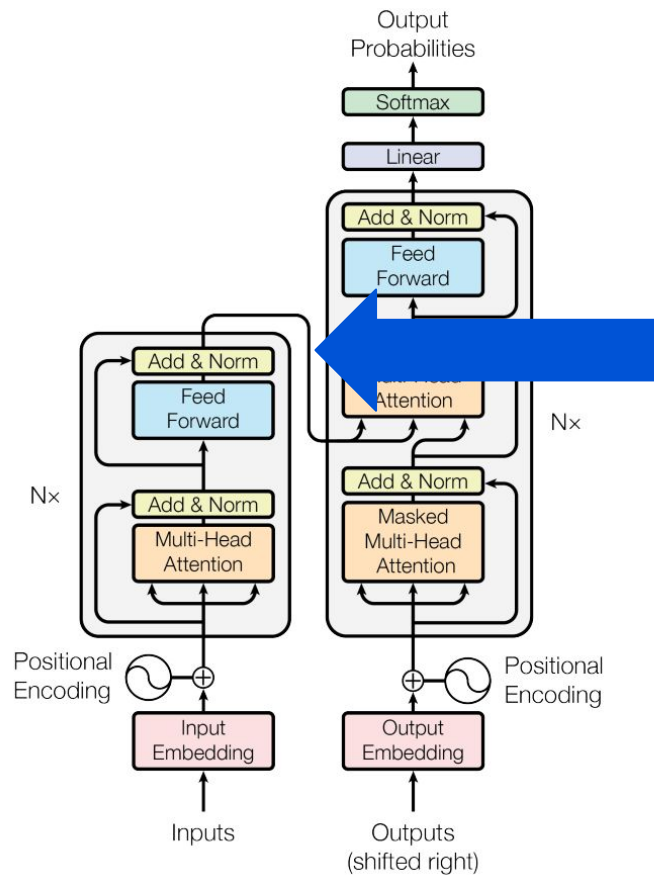
Translation architecture

Now will give details of the internals of the decoder layers and the connection between the encoder and decoder.



Translation architecture

How do we manage the information flow between the encoder and decoder?



Cross-Attention

Query comes from the decoder generating output.
The key and value come from the end of the encoder.

Query

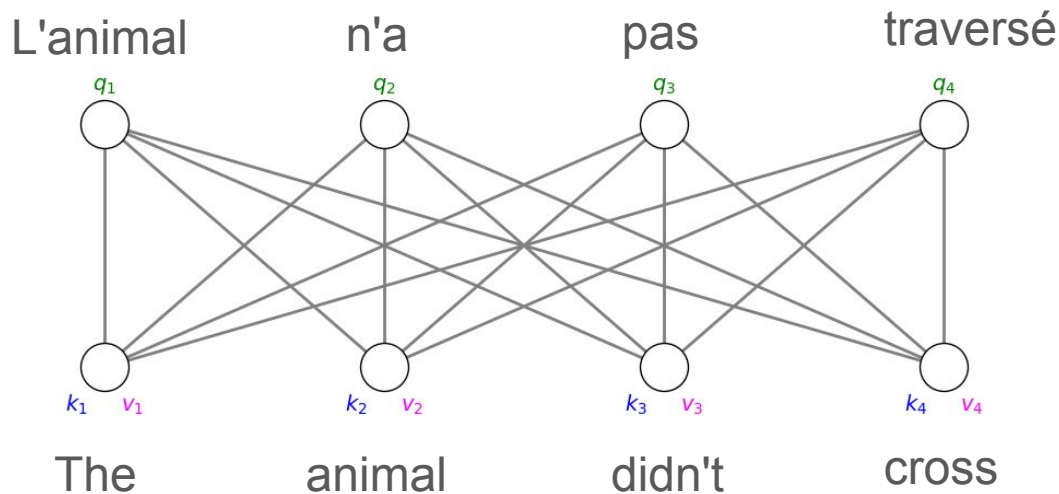
$$q_i = W_q x_i + b_q$$

Key

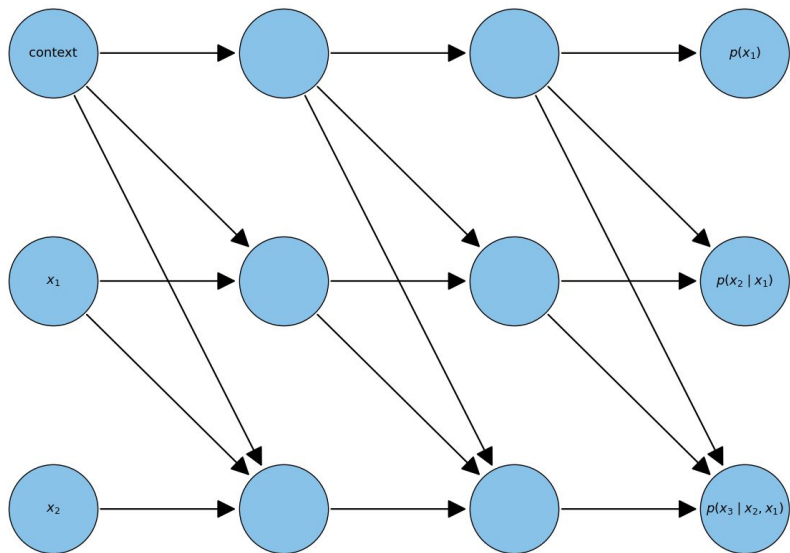
$$k_i = W_k y_i + b_k$$

Value

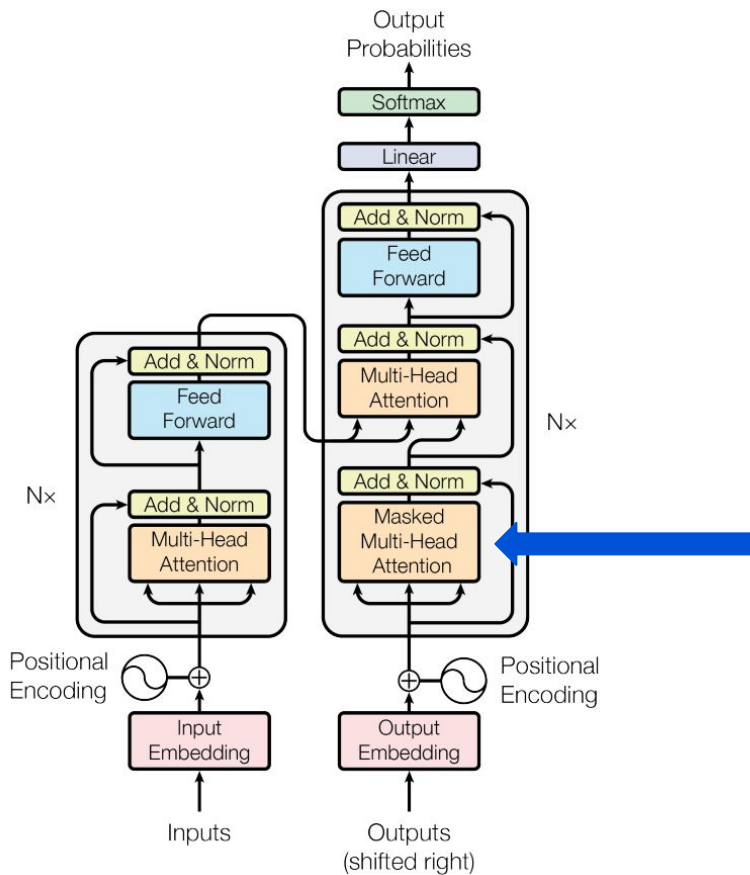
$$v_i = W_v y_i + b_v$$



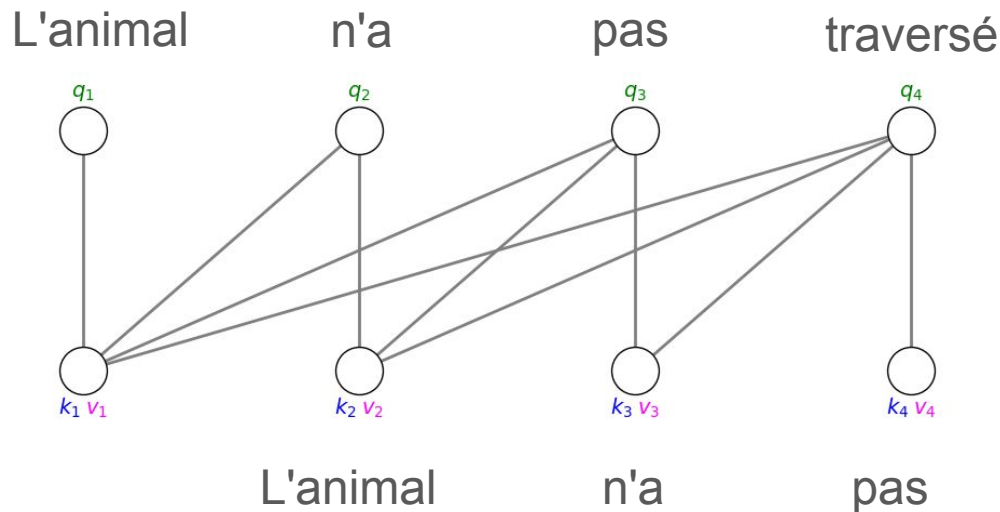
Masked self-attention



Like self-attention in the encoder except you mask connections that are out of order.



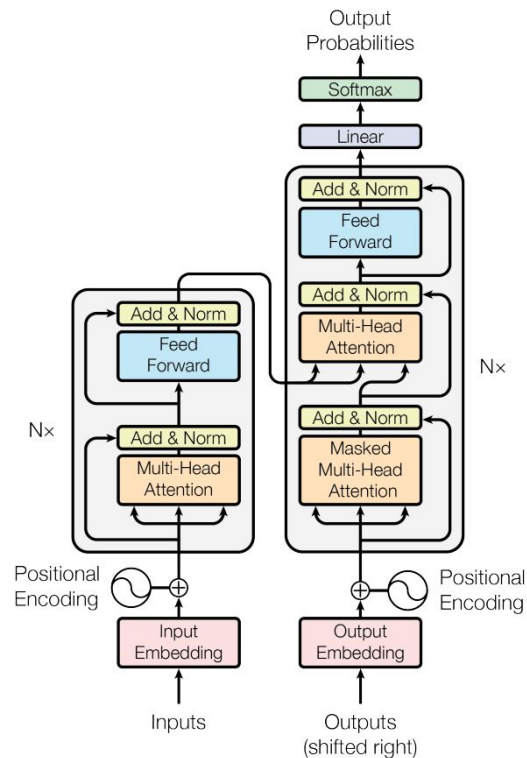
Masked self-attention



↑
HEADED
TO TOP

COMING
FROM
BOTTOM

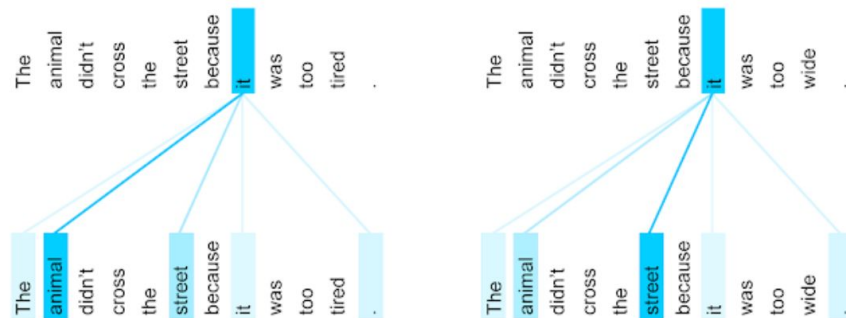
↑



Coreference resolution and attention

The animal didn't cross the street because **it** was too tired.
L'animal n'a pas traversé la rue parce qu'**il** était trop fatigué.

The animal didn't cross the street because **it** was too wide.
L'animal n'a pas traversé la rue parce qu'**elle** était trop large.



The encoder self-attention distribution for the word "it" from the 5th to the 6th layer of a Transformer trained on English to French translation (one of eight attention heads).

Vaswani et al. 2017

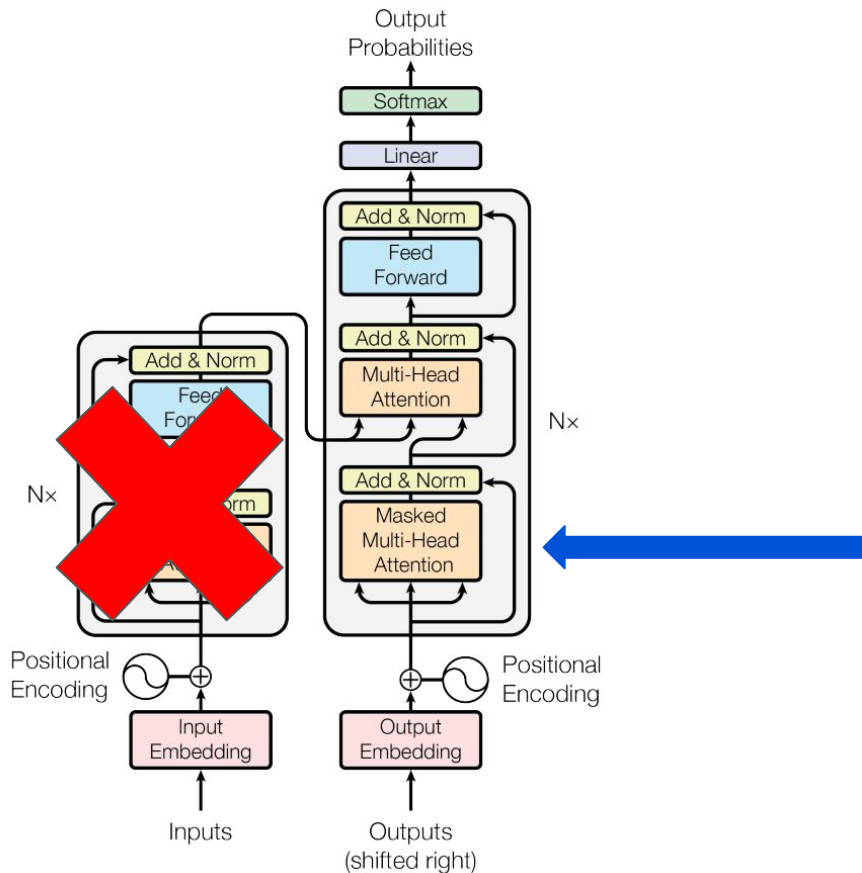
Pure generative transformers

No encoder just a decoder.

Effectively this is then sequence completion.

GPT = generative pre-trained transformer

These days can do translation pretty well “in context”.



What did sequence transformers replace?

A recurrent neural network (RNN) processes the tokens in sequence.

It has a "hidden state" that gets updated at each step as a function of the previous state and

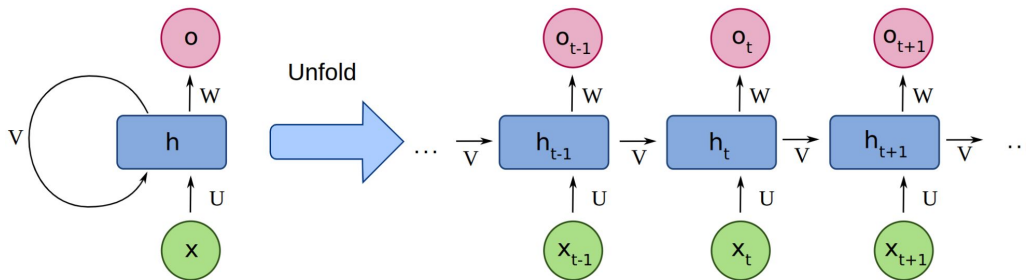
Notoriously difficult to train.

Hard to model long range dependencies.

A relative advantage is that it takes $O(T)$ flops, but conversely can't be parallelized.

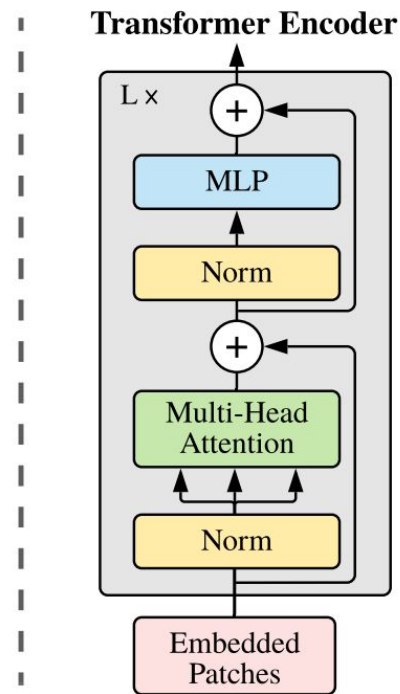
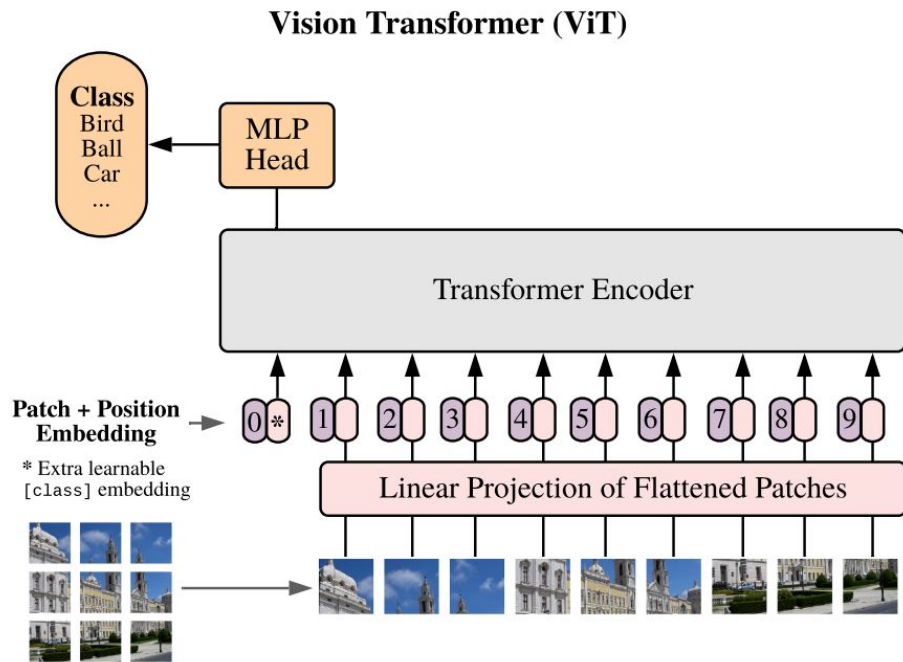
Before the transformer there were attention/RNN hybrids, hence "Attention is all you need".

Diagram shows only information flow much more complexity in reality.



-Image credit f deloche, CC BY-SA 4.0

Vision Transformers



Convolutional neural network

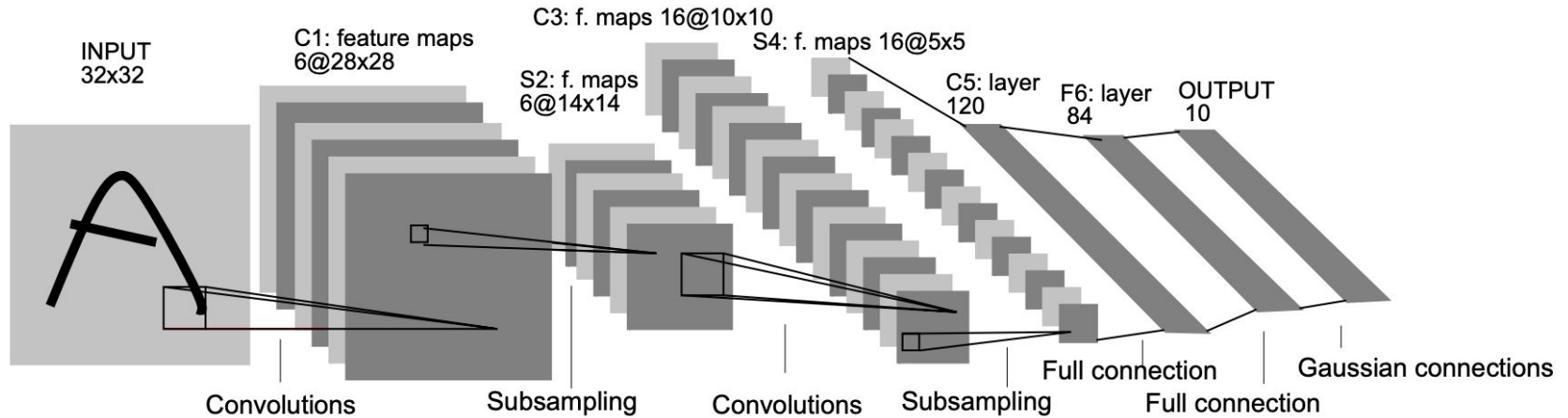


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

Incorporates translation invariance and equivariance.

Domain specifics moved from model to features

Translation model embedded time instead of having an architecture that incorporated temporal order.

Image model embedded image position instead of having an architecture that incorporated space.

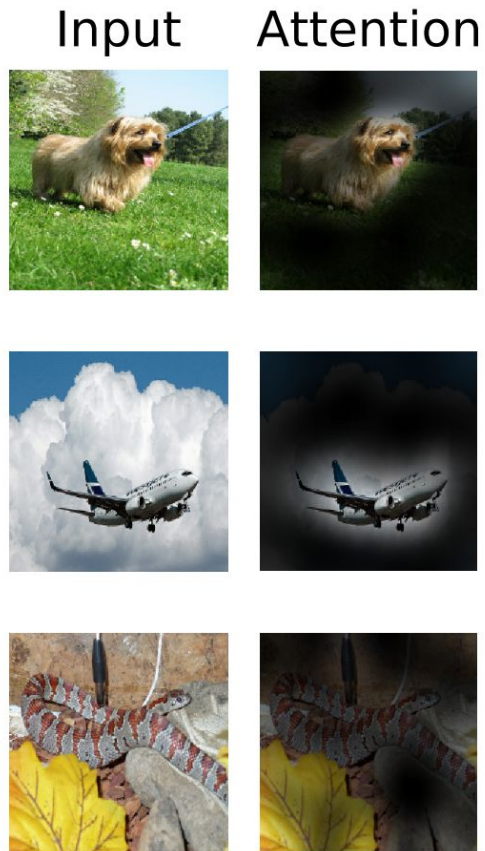
This change in paradigm is important to note. It also allows us to mix data types.

It often comes at the expense of needing more data and a higher computational cost.

Attention maps

Allow us to see where the algorithm is “looking” to make the decision on class.

Doesn't always work this cleanly!



Plan for this lecture

1) Foundational concepts

- Regression and classification
- Maximum likelihood and compression
- Multi-layer perceptrons and building representations
- Autoregression

2) Fundamentals of transformer and self-attention

- Concept of attention
- Transformer translation architecture
- Vision transformer

3) After the transformer

- Chat-GPT and dialogue agents
- Scaling
- AlphaFold II



Chat-GPT and dialogue agents

From autocomplete to an assistant

Pretraining:

- * **Very** large dataset
- * Just predict the next token.
- * Autoregressive loss



Post-training:

- * Supervised examples
- * Reinforcement learning from human feedback (RLHF).

Growth in size of pre-training data

Training data of notable LLMs



EPOCH AI | CC-BY

epoch.ai

English Wikipedia: $(7 \pm 2) \times 10^9$ tokens

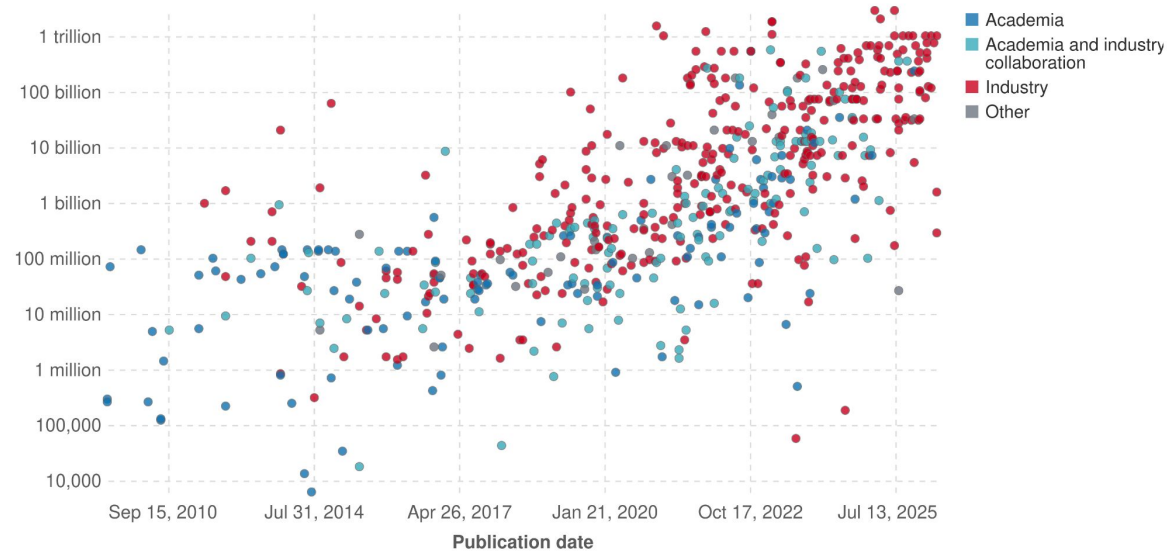
Public data is likely incomplete and large error bars

Source: <https://epoch.ai/data-insights/dataset-size-trend>

Parameters in notable artificial intelligence systems

Parameters are variables in an AI system whose values are adjusted during training to establish how input data gets transformed into the desired output; for example, the connection weights in an artificial neural network.

Number of parameters (plotted on a logarithmic axis)



Data source: Epoch AI (2026)

OurWorldinData.org/artificial-intelligence | CC BY

Note: Parameters are estimated based on published results in the AI literature and come with some uncertainty. The authors expect the estimates to be correct within a factor of 10.

Largest known is estimated at $3 * 10^{12}$

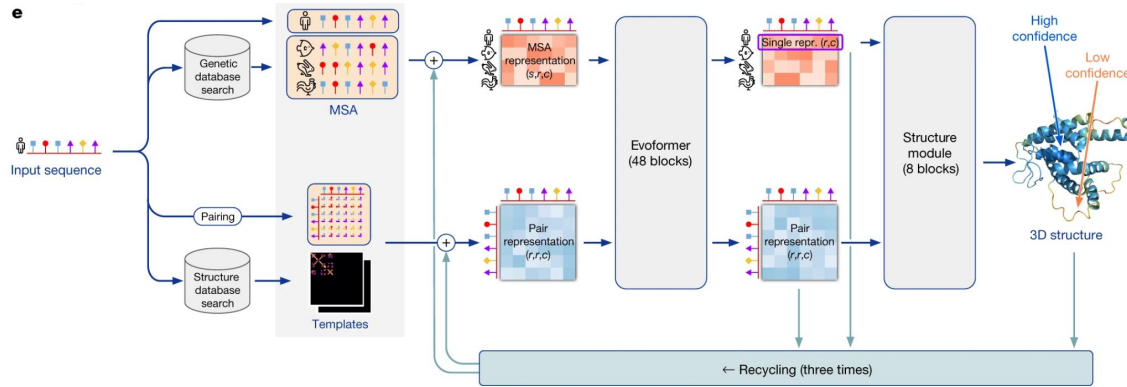
Transformer paper $2 * 10^8$

Public data is likely incomplete and large error bars.

[Source](#)

AlphaFold II

AlphaFold II architecture



Note how attention is used to integrate disparate sources of information.

Estimate about 10^8 parameters

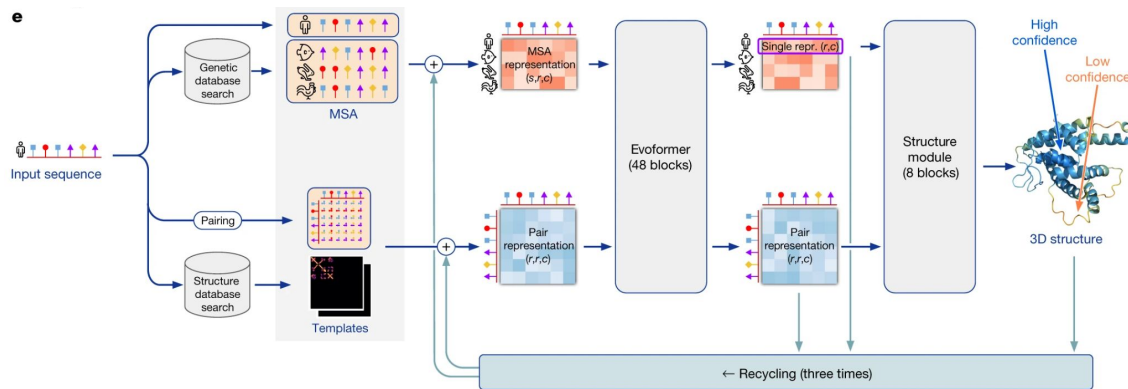
Evoformer is a structured transformer.

Structure model was replaced by a diffusion model in AlphaFold III

AlphaFold I learnt a potential to optimize which was closer to prior work.

Jumper et al. 2021

AlphaFold II architecture

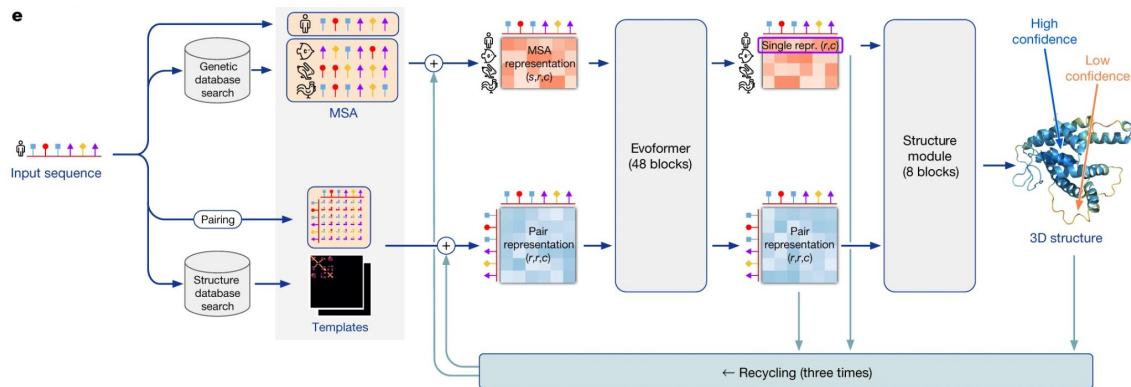


Confidence gives well calibrated uncertainty.

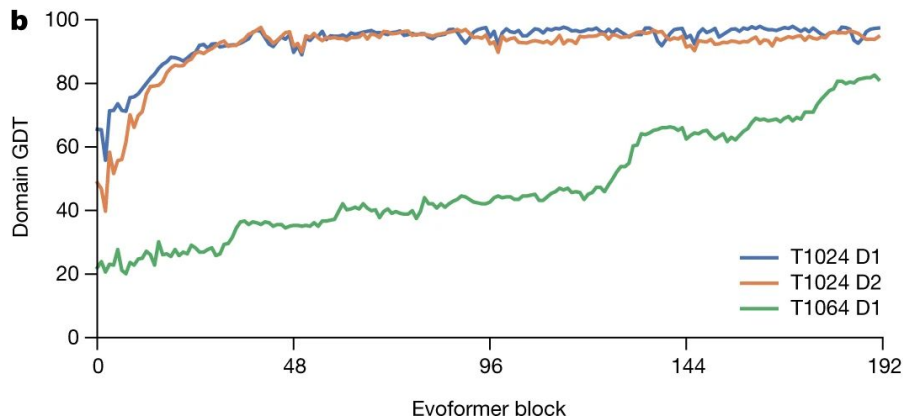
Was also used for "self-distillation" - giving much larger dataset.

Iterative progress

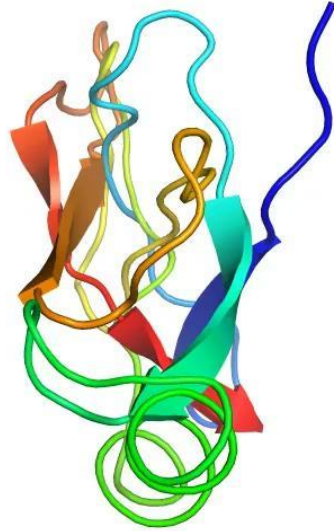
Recycling makes the network iteratively improve representation.



Plot of prediction quality as function of iterations for three different proteins.



Iterative progress



Recycling iteration 0, block 02
Secondary structure assigned from the final prediction

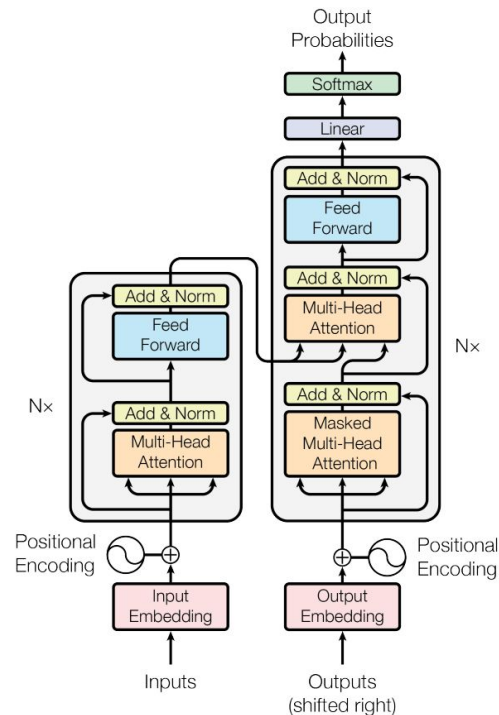
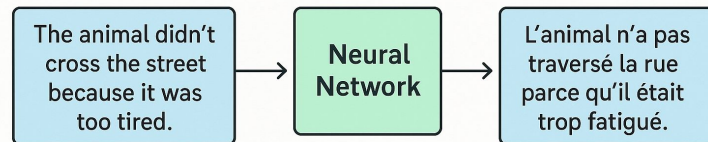
Summary

A **transformer** is a neural network **architecture** that was originally used for machine translation.

One of the innovative components was the use of particular types of **attention layers**.

The architecture is versatile. It has since been used for a huge variety of applications.

Notably a "generative" transformer is the crucial component of large language models (LLMs).



Plan for lecture series

1) **A Hitchhiker's Guide to Transformers and Attention**

Introduction to some basic concepts useful throughout the lecture series.

Introduce a crucial architecture that everyone should know.

2) **Probability transport and flow matching, with connections to diffusion models**

Crucial idea in generative modelling of natural images and science foundation models.

Beautiful mathematics with a lovely range of connections across many fields.

3) **Random neural networks, training dynamics, and scaling behaviour**

Crucial theoretical ideas in the training and scaling of neural networks.

Strong connections to ideas from statistical physics.